Vel Tech High Tech
Dr.Rangarajan Dr.Sakunthala Engineering College
Autonomous - Affiliated to Anna University
Avadi, Chennai

MICRO-LEVEL PROJECT REPORT

25ML35T / FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

TOWER OF HANOI SEARCH MODEL: ROBOTIC CONTAINER-STACKING PLANNER

SUBMITTED BY: JANAVI V

REGISTER NUMBER: 113025148038

YEAR / SECTION: II/A

SUBMITTED TO: DR. MANOJ KUMAR D S

ASSOCIATE PROFESSOR, DEPARTMENT OF CSE (AI&ML)

ACADEMIC YEAR: 2026– 2027

SUBMISSION DATE: 13-10-2026

1. ABSTRACT AND PROJECT OBJECTIVES

1.1 ABSTRACT

The Robotic Container-Stacking Planner extends the classic Tower of Hanoi search model into a practical planning problem for a robot arm that stacks shipping containers. Tower of Hanoi is first solved with Breadth-First Search (BFS) and Depth-First Search (DFS). The same problem is then expressed as a STRIPS planning domain and extended to a Blocks World with a robot arm. A Constraint Satisfaction Problem (CSP) decides which stack each container belongs to, using weight, departure order and hazard rules, with backtracking, forward checking and the MRV heuristic. A safety layer prevents the arm from lifting containers heavier than its limit, and every planned action is explained with its preconditions and effects.

1.2 Keywords

Tower of Hanoi, BFS, DFS, STRIPS Planning, Blocks World, CSP, Backtracking, Forward Checking, MRV, Robot Arm Safety.

1.3 Problem Statement

Container yards stack containers with different weights, departure times and hazard classes. A careless arrangement forces expensive re-handling or unsafe stacking, and a robot arm has a limited lifting capacity. This project generates a valid stack arrangement and a safe, explainable sequence of arm actions that builds it.

1.4 Objectives

- Solve Tower of Hanoi with BFS (optimal) and DFS and compare nodes expanded.
- Model the same problem and a container-stacking problem as STRIPS actions.
- Assign containers to stacks as a CSP respecting weight, departure and hazard constraints.
- Compare plain backtracking with forward checking and forward checking + MRV.
- Enforce a robot-arm weight limit and explain every action in the final plan.

1.5 Scope

The project covers state-space search on Tower of Hanoi, STRIPS planning for Hanoi and Blocks World, CSP-based stack assignment, arm safety checks, explanations and small comparison experiments. It is a simulation; it does not control physical hardware.

2. INTRODUCTION AND PROPOSED SOLUTION

2.1 Background

Tower of Hanoi moves disks between three pegs so that a larger disk never rests on a smaller one. Its stacking rule is the same kind of rule found in container yards, where heavier containers must sit below lighter ones. This makes Hanoi a natural starting model for a stacking planner.

2.2 Existing Approach

Containers are commonly stacked manually or in arrival order, and the arm is told only where to move each item.

2.3 Limitations

- Arrival-order stacking ignores weight and departure order, causing re-handling.
- Plans without safety checks may ask the arm to lift items beyond its capacity.
- Plans without explanations are hard for operators to verify.

2.4 Proposed Solution

The planner first decides the goal stacks with a CSP, then builds a STRIPS Blocks World problem from the current and goal stacks, finds a plan with BFS over fact sets, validates each step against the arm limit, and prints a reason for every action.

2.5 Users and Benefits

Port and warehouse planners and students of AI planning can use it to see how search, planning and constraint solving combine in one pipeline.

3. AI CONCEPTS AND PROBLEM FORMULATION

3.1 AI Concepts

Uninformed state-space search (BFS, DFS), STRIPS automated planning, Constraint Satisfaction Problems, heuristic search (MRV), forward checking, and explainable rule-based reasoning.

3.2 Suitability

BFS guarantees the shortest Hanoi solution; STRIPS gives a general action language with preconditions and effects; CSP naturally captures stacking rules; MRV and forward checking reduce the search effort.

3.3 Problem Formulation

Element	Description
Initial state	All disks on peg A (Hanoi); all containers on the table (container stacking).
Actions	Move(disk, src, dst) for Hanoi; Pickup, Putdown, Stack, Unstack for the robot arm.
Transition model	Delete and add lists of each STRIPS action update the set of true facts.
Goal	Disks on peg C; or containers arranged in the CSP-chosen goal stacks.
Path cost	One per action; BFS returns the minimum number of actions.

3.4 PEAS

Component	Description
Performance	Valid stacks, shortest plan, no overweight lift, no hazardous pairs.
Environment	Three pegs or stacks, a set of containers, one robot arm.
Actuators	Robot arm actions: pick up, put down, stack, unstack.
Sensors / Inputs	Container weight, departure order, hazard flag, stack capacity, arm limit.

3.5 Inputs and Outputs

Inputs are the container manifest (weight, departure, hazardous), the number of stacks, maximum stack height and the arm weight limit. Outputs are the stack assignment, the action plan, search statistics and per-step explanations.

4. SYSTEM ARCHITECTURE AND DESIGN

4.1 Architecture Diagram

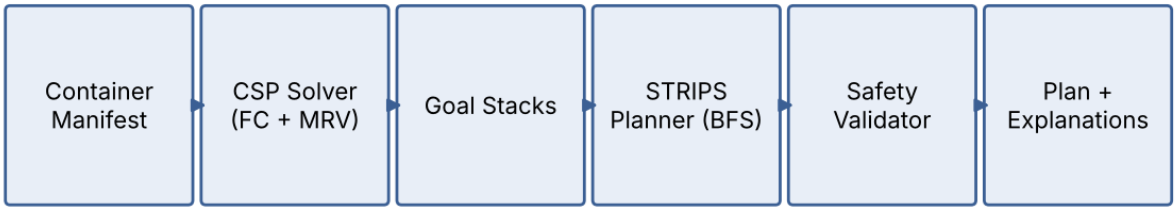


Figure 1. System pipeline of the Robotic Container-Stacking Planner.

4.2 Data Representation

A Hanoi state is a tuple of three peg tuples. A planning state is a frozenset of facts such as On(x,y), OnTable(x), Clear(x), Holding(x) and ArmEmpty. An Action is an immutable record with name, arguments, preconditions, add list and delete list. A Container records name, weight, departure order and hazard flag.

4.3 Modules

Module	Responsibility
TowerOfHanoi	Generates legal moves; runs BFS or DFS and returns the path and nodes expanded.
plan / Action	Generic STRIPS planner: BFS over fact sets.
hanoi_domain / blocks_domain	Build initial facts, goal facts and ground actions; blocks_domain drops actions that lift overweight items.
solve (CSP)	Assigns containers to stacks with backtracking, forward checking and optional MRV.
show_robot_plan	Re-checks safety, prints explanations and verifies the goal.

4.4 Tools

Python 3 standard library only (random, time, collections.deque, dataclasses); VS Code.

5. ALGORITHM AND METHODOLOGY

5.1 Methodology

The number of disks is derived from the register number in the code (3 + digit-sum mod 3). Hanoi is searched with BFS and DFS using a seeded shuffled move order. Then the STRIPS planner solves Hanoi and Blocks World. Finally a CSP produces goal stacks, and the planner builds a safe arm plan to reach them.

5.2 Search and Planning Rules

Hanoi move rule: a disk may move from src to dst only if src is non-empty and dst is empty or has a larger top disk. STRIPS action applicable if Pre $\subseteq$ State; result = (State − Delete) $\cup$ Add. Plan cost = number of actions. The minimum number of moves for n disks is $2^n - 1$.

5.3 CSP Algorithm

1. Create one variable per container; the domain is the set of stack indices.
2. Reject early if containers exceed total capacity or hazardous containers exceed the number of stacks.
3. Choose a variable (first unassigned, or the one with the smallest domain under MRV).
4. Try each stack that is consistent: below max height and compatible with every container already in it.
5. Apply forward checking: remove that stack from the domain of other containers it no longer fits; stop if a domain becomes empty.
6. Backtrack on failure and restore the removed values.

5.4 Constraints and Safety Rules

- Two hazardous containers may not share a stack.
- Containers in one stack must be ordered consistently: $(w_1 - w_2)(t_1 - t_2) \geq 0$, so heavier containers go below and later-departing ones sit lower.
- A stack may not exceed the maximum height.
- Pickup and Unstack actions are not generated for any container heavier than the arm limit.
- A second runtime check raises a safety stop if any plan step lifts an overweight container.

5.5 Search Comparison

BFS expands states level by level and is optimal for unit costs; DFS uses less memory but is not guaranteed to be optimal. Plain backtracking, forward checking and MRV are compared by counting assignments tried and measuring runtime on identical inputs. All figures reported in Section 7 come from an actual run of the program.

6. IMPLEMENTATION AND GITHUB

6.1 Environment

Python 3 running in VS Code. No external libraries are required.

6.2 Implementation Steps

1. Implement TowerOfHanoi with legal-move generation, BFS and DFS.
2. Implement the Action record and the STRIPS planner with explanations.
3. Create hanoi_domain and blocks_domain with input validation and arm-limit filtering.
4. Implement the Container class and the CSP solver with forward checking and MRV.
5. Add demonstration functions that compare methods and run the full pipeline.

6.3 Execution

Open the project folder in VS Code and run the following command. The program prints all results in the console.

```
python main.py
```

6.4 Commit History

Add the repository link and the commits actually made here, for example: “add Hanoi BFS/DFS”, “add STRIPS planner”, “add CSP solver with MRV”.

7. RESULTS AND OBSERVATIONS

All values below were produced by running the submitted program (REG_NO = 15217 in the code, giving 4 disks).

7.1 Tower of Hanoi: BFS vs DFS

Method	Moves	Nodes expanded	Note
BFS	15	81	Optimal ($2^4 - 1$); explored all $3^4 = 81$ states
DFS	15	16	Found a 15-move path here; optimality is not guaranteed
STRIPS (BFS)	15	68	Same plan length as BFS

7.2 Blocks World Scaling (all blocks on table → one tower)

Blocks	Plan length	Nodes expanded
3	4	19
4	6	108
5	8	771

Plan length grows linearly ($2(n - 1)$ actions) while expanded nodes grow sharply, showing the cost of uninformed planning.

7.3 CSP Comparison

Containers	Stacks	Height	Plain tried	FC tried	FC+MRV tried	Plain (s)	FC (s)	MRV (s)
12	4	3	12	12	12	0.0001	0.0001	0.0001
18	5	4	603	264	18	0.0014	0.0008	0.0001
18	6	3	590	419	18	0.0013	0.0011	0.0001

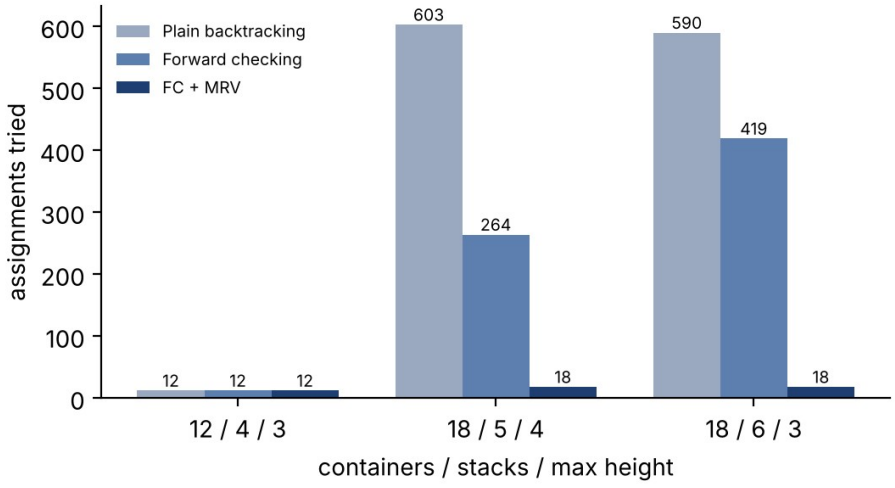


Figure 2. Assignments tried by each CSP method (lower is better).

All three methods solved every instance. Forward checking cut assignments by about 56% on the 18-container, 5-stack case, and FC + MRV needed only 18 assignments (one per container, no backtracking). Runtimes are tiny and may vary between machines.

7.4 End-to-End Example

Container	Weight	Departure	Hazardous
C1	2	4	No
C2	5	5	No
C3	3	1	No
C4	9	3	Yes
C5	7	6	No
C6	1	2	No

CSP result (3 stacks, height 3, bottom → top): [C5, C2, C1], [C4, C3], [C6]; 6 assignments tried, 0 backtracks. Arm limit = 9, so all containers are liftable. The planner produced a 6-action plan (1719 states expanded):

Step	Action	Reason (preconditions → effect)
1	Pickup(C2)	ArmEmpty, Clear(C2), OnTable(C2) → Holding(C2)
2	Stack(C2, C5)	Holding(C2), Clear(C5) → On(C2,C5)
3	Pickup(C1)	ArmEmpty, Clear(C1), OnTable(C1) → Holding(C1)
4	Stack(C1, C2)	Holding(C1), Clear(C2) → On(C1,C2)
5	Pickup(C3)	ArmEmpty, Clear(C3), OnTable(C3) → Holding(C3)
6	Stack(C3, C4)	Holding(C3), Clear(C4) → On(C3,C4)

The program verified that all required stack relations hold at the end. C5, C4 and C6 stay where they are because they are already on the table at the base of their stacks.

7.5 Test Cases

Test	Scenario	Expected result	Observed
1	Hanoi with 4 disks	Shortest plan = 15 moves	BFS and STRIPS both 15
2	Blocks World, 3–5 blocks	Valid tower plan	Plans of 4, 6, 8 actions
3	Hazardous containers	Never share a stack	C4 alone in its stack
4	Heavy before light, early departure on top	Consistent stack order	Satisfied in all stacks
5	Container heavier than arm limit	No Pickup/Unstack generated	Enforced by blocks_domain and runtime check

8. DISCUSSION, LIMITATIONS AND FUTURE WORK

8.1 Discussion

BFS provides optimality, STRIPS provides a reusable action language, CSP decides the goal, and safety checks make the plan trustworthy. The Hanoi result shows DFS can be faster on a given instance, but only BFS guarantees the shortest plan.

8.2 Limitations

- BFS over fact sets grows quickly with the number of containers (771 nodes at 5 blocks; 1719 for the 6-container example).
- The arm model is simple: it holds one container and has a single weight limit.

- The CSP stack order is derived from weight and departure only; no stack geometry or crane movement cost is modelled.
- Results use small, randomly generated instances.

8.3 Future Enhancements

- Use heuristic search (A*) to reduce planning nodes.
- Add multiple arms, travel distance and time costs.
- Add a graphical simulation of the yard.
- Test on larger real manifests.

8.4 Applications

Port terminals, warehouses, automated storage and AI planning education.

8.5 Learning Outcomes

The project demonstrates state-space search, STRIPS planning, constraint satisfaction with heuristics, safety constraints and explainable plans.

9. CONCLUSION AND REFERENCES

9.1 Conclusion

The project shows how the Tower of Hanoi search model grows into a robotic container-stacking planner. BFS finds the optimal 15-move Hanoi solution, STRIPS planning generalises it to Blocks World, and a CSP with forward checking and MRV chooses valid stacks with far fewer assignments than plain backtracking. Arm-limit filtering and explained actions keep the resulting plan safe and verifiable.

9.2 References

- Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach.
- Fikes, R. and Nilsson, N., “STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving”, Artificial Intelligence, 1971.
- Python documentation: https://docs.python.org/3/
- Provided Foundations of AI micro-project report template.